

Introduction to Aerial Robotics

Lab Tutorial 2 (Project 1 Phase 4)



Professor: Shaojie Shen

TAs: Yang Xu, Pusen Gao

Dept. of ECE, HKUST

25 Mar. 2026

Logistics of Project 1 Phase 4

- Lab 1 (Last week): Drone assembling and manual flight
- **Lab 2 (Today – 22 April):** Development environment and fly in API mode with OptiTrack
- Lab 3 (29 April – 13 May): State estimation and trajectory planning by your own algorithm

These days' Objective

- Get familiar with software configuration onboard
- Hover automatically
- Write a trajectory defined by yourselves and fly it automatically
- Submit a report of these days' work via Canvas, including experiment images, tracking error calculation, etc.

0. Prerequisite Knowledge

- Linux basis
 - Terminal and some common commands
 - Packages and file management system on Linux
- ROS basis



1.1 Install Robot Operating System (ROS)

- Open your terminal

1. `sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'`
2. `sudo apt install curl` # if you haven't already installed curl
3. `curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add -`
4. `sudo apt update`
5. `sudo apt install ros-noetic-desktop-full`
6. `source /opt/ros/noetic/setup.bash`
7. `echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc`
8. `source ~/.bashrc`
9. `sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator python3-wstool build-essential`
10. `sudo apt install python3-rosdep`
11. `sudo rosdep init`
12. `rosdep update`

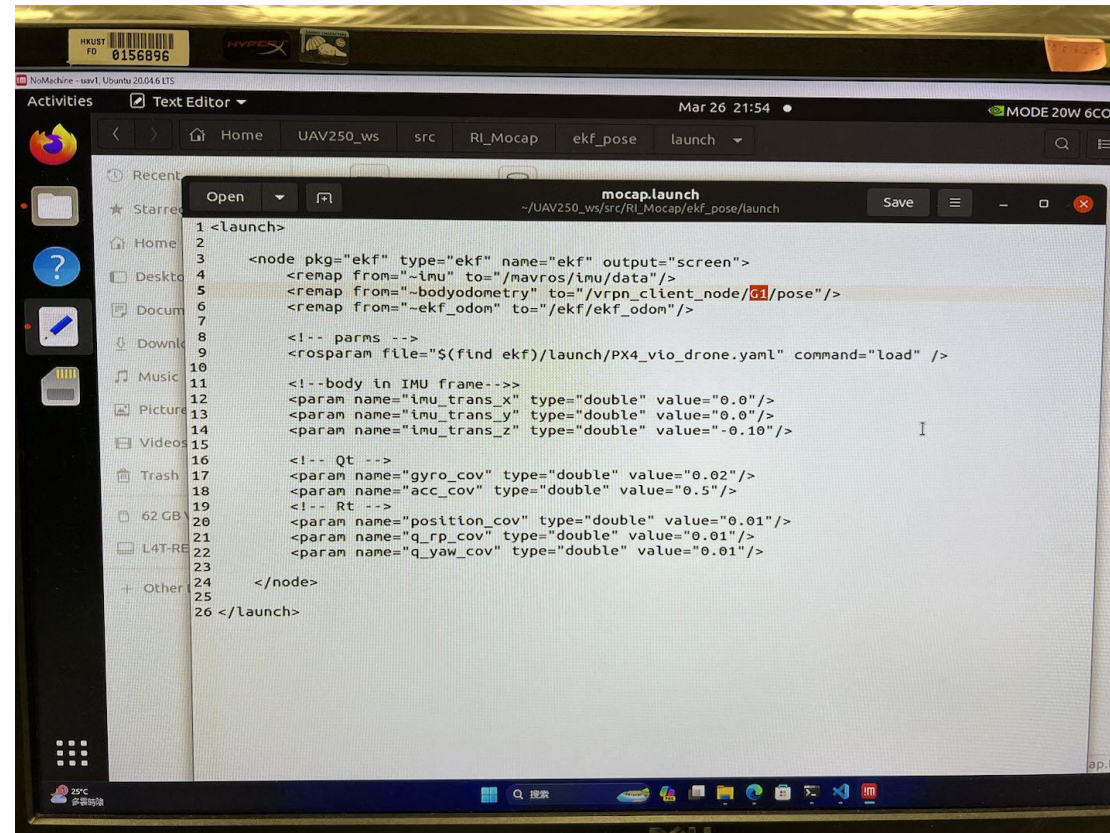
1.2. Install MAVROS

- Open your terminal

1. `sudo apt-get install ros-noetic-mavros ros-noetic-mavros-extras ros-noetic-mavros-msgs`
2. `wget`
https://raw.githubusercontent.com/mavlink/mavros/master/mavros/scripts/install_geographiclib_datasets.sh
3. `chmod +x ./install_geographiclib_datasets.sh`
4. `sudo ./install_geographiclib_datasets.sh`

1.3. Modify RI_Mocap

- Find the mocap.launch in the given RI_Mocap package, then modify the group ID as your group.

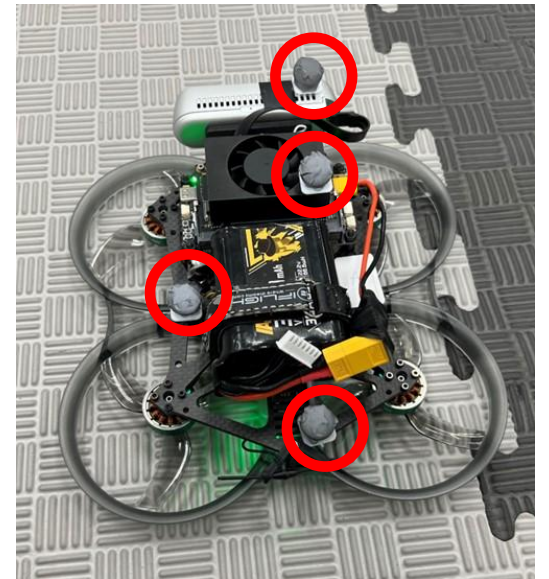


The screenshot shows a Linux desktop environment with a text editor open to the file `~/UAV250_ws/src/RI_Mocap/ekf_pose/launch/mocap.launch`. The file contains ROS launch XML code. The code defines a `launch` group with a single `node` of package `ekf`. It includes remappings for `imu`, `bodydometry`, and `ekf_odom`. It also sets parameters for IMU frame transformations and covariance matrices. The code is as follows:

```
1<launch>
2
3  <node pkg="ekf" type="ekf" name="ekf" output="screen">
4    <remap from="/imu" to="/mavros/imu/data"/>
5    <remap from="/bodydometry" to="/vrpn_client_node/61/pose"/>
6    <remap from="/ekf_odom" to="/ekf/ekf_odom"/>
7
8    <!-- params -->
9    <rosparam file="$(find ekf)/launch/PX4_vto_drone.yaml" command="load" />
10
11    <!-- body in IMU frame -->
12    <param name="imu_trans_x" type="double" value="0.0"/>
13    <param name="imu_trans_y" type="double" value="0.0"/>
14    <param name="imu_trans_z" type="double" value="-0.10"/>
15
16    <!-- Qt -->
17    <param name="gyro_cov" type="double" value="0.02"/>
18    <param name="acc_cov" type="double" value="0.5"/>
19    <!-- Rt -->
20    <param name="position_cov" type="double" value="0.01"/>
21    <param name="q_rp_cov" type="double" value="0.01"/>
22    <param name="q_yaw_cov" type="double" value="0.01"/>
23
24  </node>
25
26</launch>
```

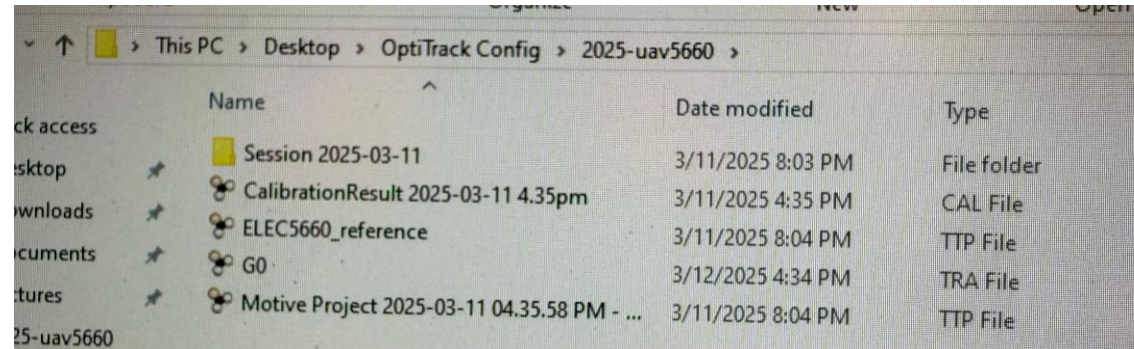

2. Motion Capture (OptiTrack System)

- This can be used as ground truth for your project 1 phase4.
- Install OptiTrack 'sudo apt-get install ros-noetic-vrpn' in your NX.
- **This CANNOT be used in your EKF.**
 - We will test your result outside the range of OptiTrack
- Stick OptiTrack markers on your drone
 - At least 4 non-symmetric markers
 - Don't stick them the same way for each group.



2. Motion Capture (OptiTrack System)

- Copy and rename a ELEC5660_Gx.ttp file from the ELEC5660_reference.ttp
 - X is your group number
- Save your group's config file under the folder below

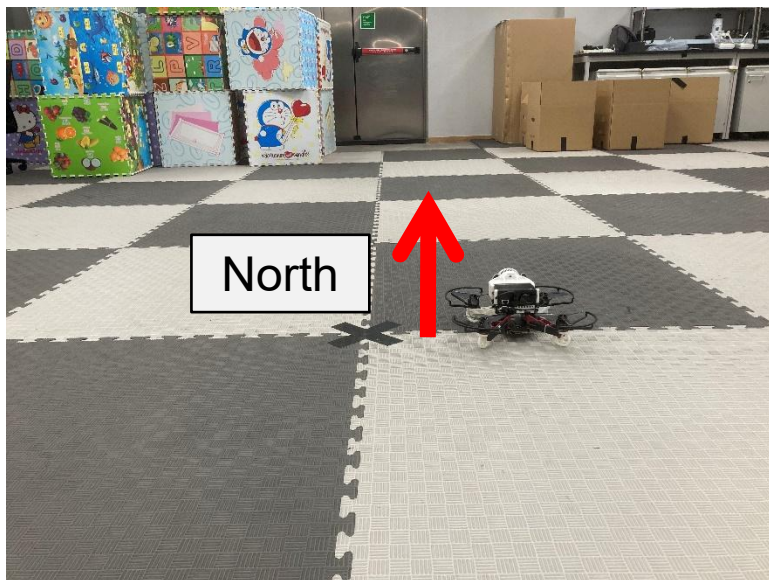


- **!!! DO NOT MODIFY OTHERS CONFIG AND USING WROING CONFIG WILL MAKE YOUR DRONE CRASH**

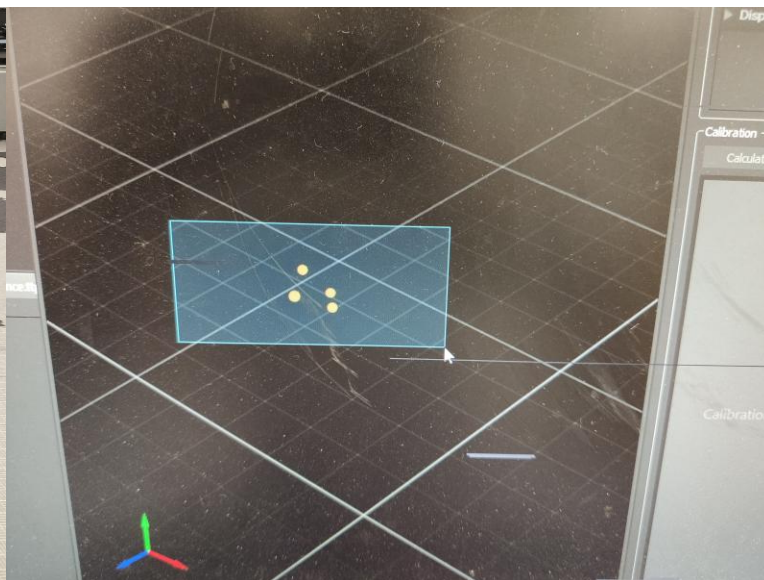
2. Motion Capture (OptiTrack System)

- Create a rigid body for your drone
 - Put your quadrotor in the field, head north
 - Drag-select all markers on your drone, create a rigid body
 - Change name and user ID to your group number
 - e.g., for Group 3, using name G3 and user ID 3
 - You need to delete old one and recreate your rigid body if you restick any OptiTrack marker

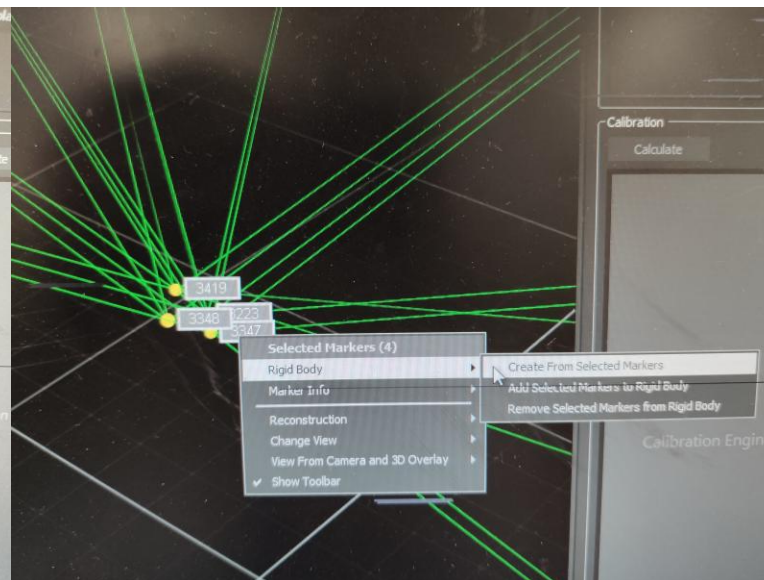
2. Motion Capture (OptiTrack System)



Head north



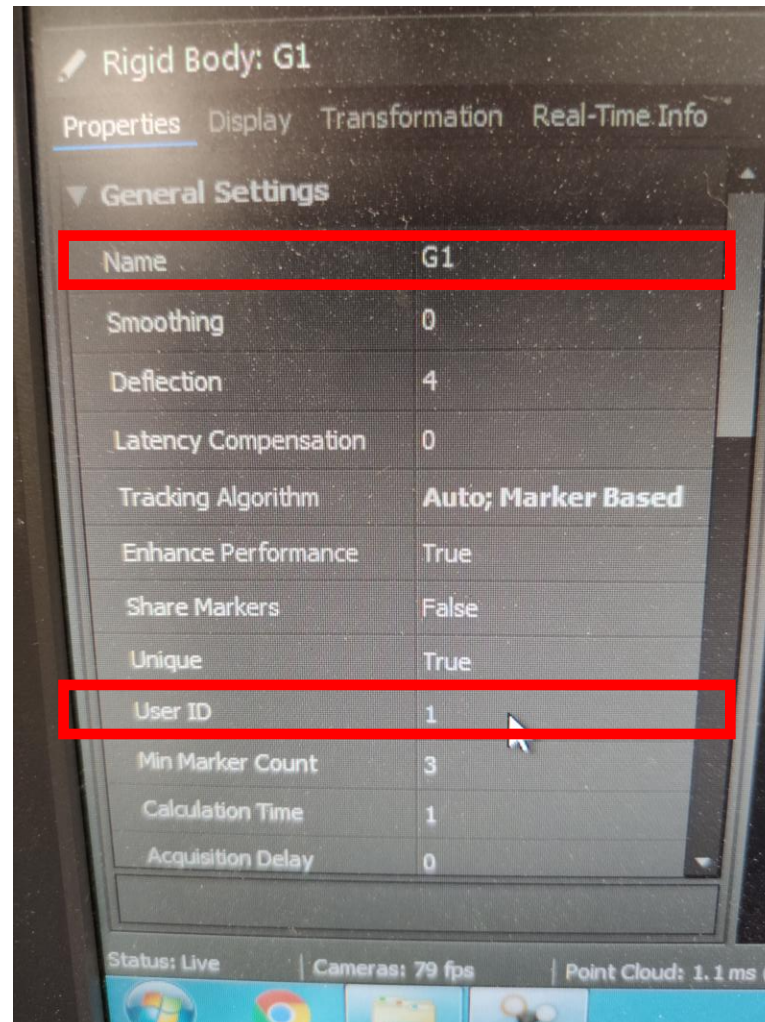
Drag-select



create rigid body

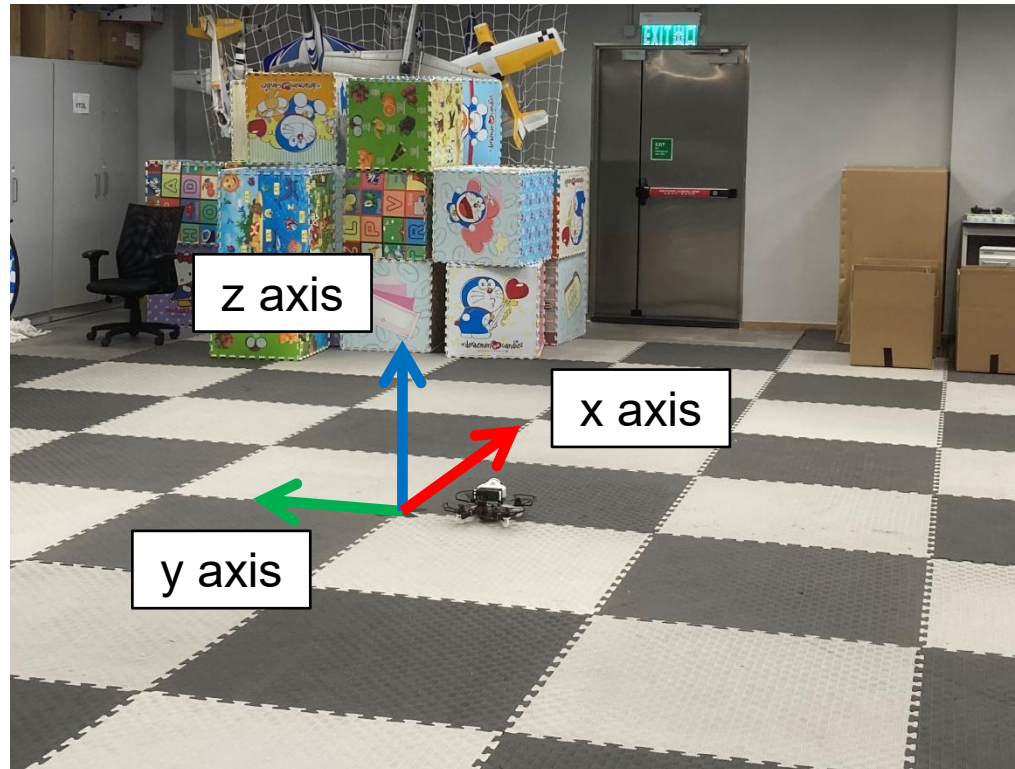
2. Motion Capture (OptiTrack System)

- Properties
 - Name: Gx
 - ID: x
 - x: group number



3. OptiTrack Related

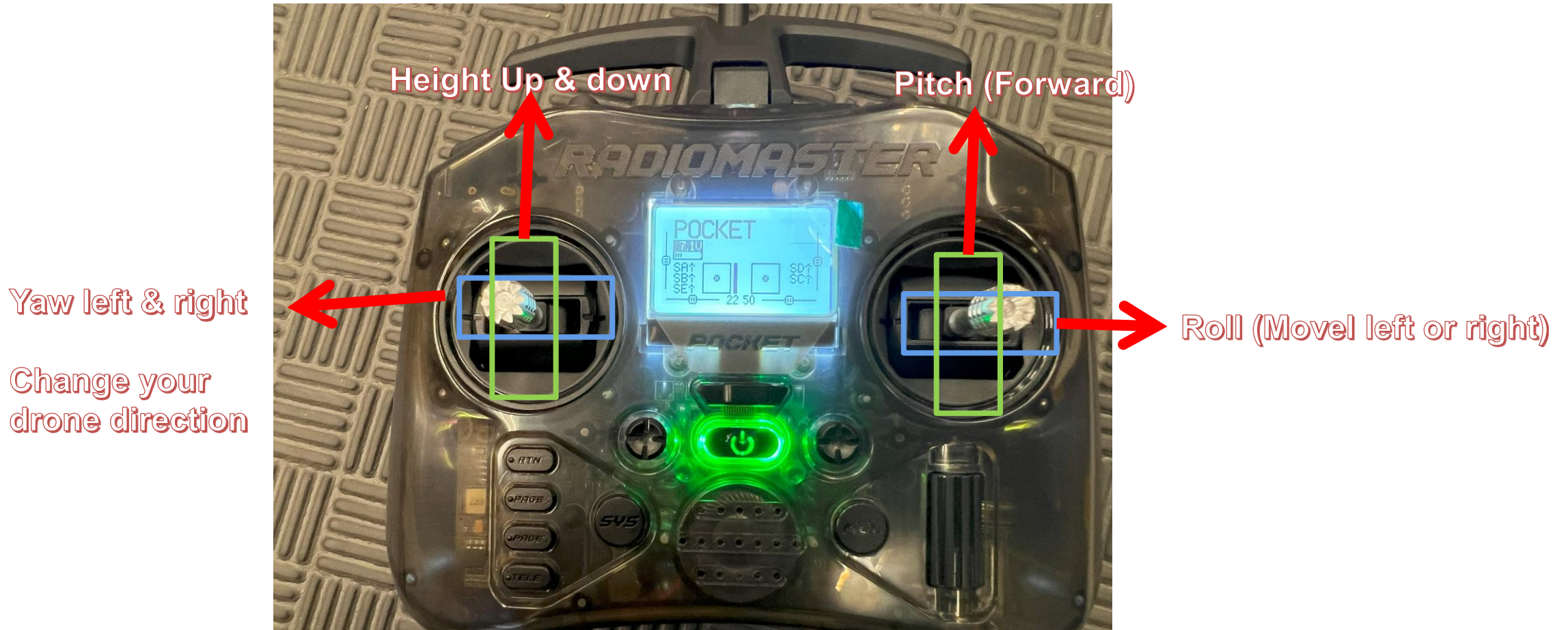
- Coordinate system



4. Drone Controller

- px4_ctrl:
 - A ROS package for automatic drone position control.
- RI_Mocap
 - A ROS package for Optitrack.

Radio Controller



We recommend you do not change the yaw. It is much friendly to beginner

Radio Controller



5. Launch the code

- Run your code in “/home/\${YOUR_DRONE}/uav_ws”
 - source devel/setup.bash
 - ./mocap_run.sh

Tips: Dry run all the pipeline on the ground!!!

First you need switch the mode, you will see
MANUAL_CTRL(L1) --> AUTO_HOVER(L2)

Then you can trigger the trajectory, you will see
TRIGGER sent, allow user command.

Tips

- Tips: record data:
 - If you do not have the image data, you can record all topics.
 - `rosbag record -a -tcpsnodelay`
- Tips: Visualize data use `rviz`

TODO

- **TODO:** Add your trajectory by modifying the `traj_generator/src/traj_test.cpp` to set your own trajectory.
 - Tips: You can change the waypoint in the cpp.
 - Tips: You can easily upload files to NX by direct drag-and-drop upload via nomachine.
- You can also fine tune the controller parameters to obtain better performance.

```
--  
40 gain:  
41     # Cascade PID controller. Recommend to read the code.  
42     Kp0: 2.5  
43     Kp1: 2.5  
44     Kp2: 2.5  
45     Kv0: 2.0  
46     Kv1: 2.0  
47     Kv2: 1.5  
--
```

```
ctrl_param_fpv.yaml  
~/projects/UAV250_ws/src/px4ctrl/config
```

Before Fly in API Mode

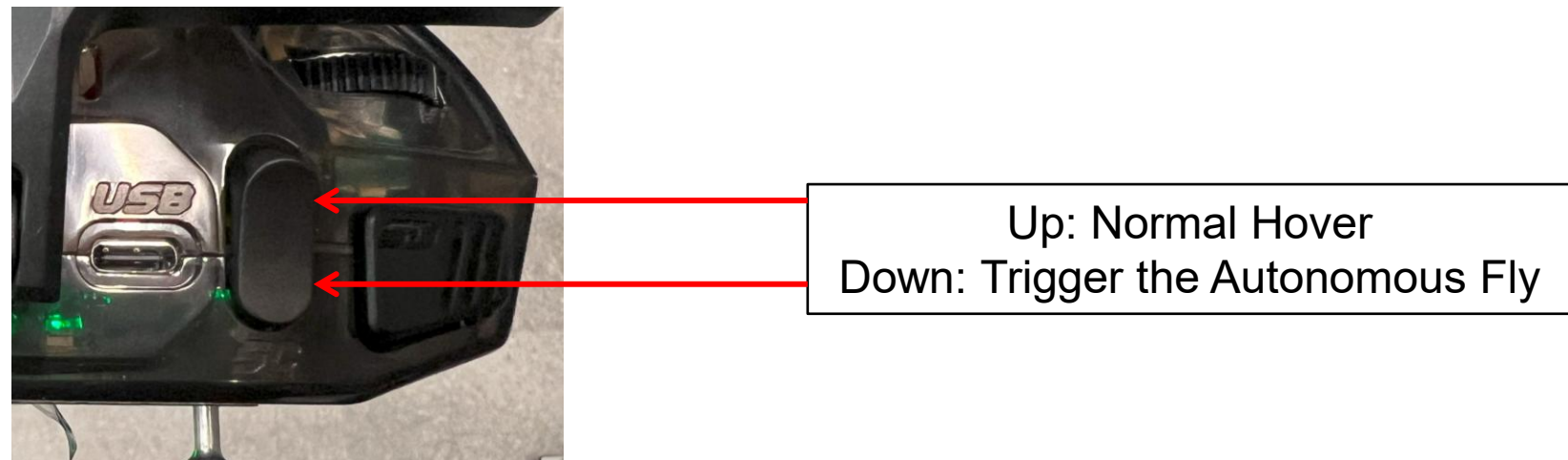
- Make sure NO problem with manual flight
- Check battery
- Check your Odometry topic (OptiTrack or EKF in next lab)
 - In Optitrack, you can use `rostopic echo /ekf/ekf_odom`

Flight in API Mode (**After take-off**)

- Switch to API mode



- Switch to onboard command mode



Suggested Workflow

- Test your drone control in hover mode
- Test your drone control with provided functions (e.g. circle, line, etc.)
- Run your own trajectory in manual mode, check values
- Test your own trajectory in API mode
- **!!! Anything abnormal during flight, switch back to manual control and land immediately**
- **!!! Clear the flying field if you are going to fly your drone**

If You Crash ...

- Stop motor immediately by emergency stop
- Unplug battery immediately (after motor stop spinning)
- Check if any damage
 - Fix them by yourself and ask TA for backup materials (if any)
- May need to re-calibrate IMU after crash
- Fly in manual mode again to check if everything works

Enjoy Flight

- Note again: Be careful during your experiments because your robot cost more than **HK\$ 5,000 !!!**

